

PROG7314

POE Part 1

Planning & Design



Group 8

ST10084625 – Caleb Keanu Naidoo

ST10441678 – Taytem Pillay

ST10460086 – Reece Mikhael Moodley

Word Count: 2150

Table of Contents

Introduction.....	pg. 3
App Overview.....	pg. 3 – 5
Detailed List of Requirements.....	pg. 6 – 7
User Interface Design.....	pg. 8 – 11
Rest API and Back-End Architecture.....	pg. 12 – 16
Project Plan.....	pg. 17 – 21
Conclusion.....	pg. 21
References/Bibliography.....	pg. 22

Matrix Market Planning and Design Report

Introduction

Digital marketplaces have made it easier for users to buy, sell and discover products and services within their local communities. However, the research conducted for this project has identified a significant gap between general-purpose marketplace platforms and the specific needs of university students. Tertiary students have to operate within academic environments with specific schedules, limitations, requirements and academic resource sharing.

Matrix Market is a mobile marketplace application specifically designed for university students and campus communities. The application's aim is to provide a trusted environment where students can participate in peer-to-peer commerce, share academic resources, and connect through structured skill and service opportunities. Matrix Market focuses on addressing the specific weaknesses identified through the comparative research. This Planning and Design Document will define the application's functional and non-functional requirements, system architecture, database and data-management design, user interface and the overall implementation strategy of the application. Design models and specifications will also be shown to demonstrate how the proposed system will operate, as well as how the individual components will interact with each other.

Application Overview

Official Name of the App: Matrix Market

Matrix Market is a native Android marketplace application designed specifically for tertiary students and university campus communities. This application's aim is to provide students with a trusted environment for peer-to-peer buying and selling, academic resource sharing, skill or service collaboration, as well as provide an overall engaging marketplace experience.

The application will incorporate academic-based categorisation, institutional identity verification, barcode-assisted textbook identification, real-time communication, offline functionality, as well as a gamified reputation system. These features are intended to differentiate Matrix Market from both general-purpose marketplaces, as well as any other already existing student-focused platforms.

Initial Application Icon Design



Figure 1: Initial App Icon PNG Asset

The Matrix Market application icon uses a minimalistic black and gold colour scheme. The gold letter “M” provides an immediate visual association with the name “Matrix Market”. The bold and simple design also allows the application to remain recognisable when displayed on a mobile device.

The colour palette chosen represents professionalism, value and quality and the icon is minimalistic and modern in order to fit the clean user interface of the application.

Innovative Features

- **Institutional Identity Verification**

Matrix Market will implement a student verification process in order to improve trust and reduce fake or unverified accounts. (Africa, 2026).

- **Academic Marketplace Taxonomy**

Matrix Market will use a student-specific academic taxonomy. Therefore, listings can be structured around information/filters like university campus, module code, academic category, and other relevant student-focused components (Africa, 2026).

- **ISBN Barcode Scanner**

Matrix Market will incorporate a camera-based ISBN barcode scanning feature using ML Kit, so students will be able to scan the barcode of a physical textbook which will be used to assist with identifying the textbook and connecting it with relevant academic information (Australia, 2026).

- **Real-Time In-App Communication**

Matrix Market will provide real-time in-app communication between buyers and sellers through a proposed WebSocket and Firebase-based chat pipeline. This will allow students to discuss listings, negotiate prices, ask questions, etc.

- **Offline-First Marketplace Functionality**

The application will use a Room local database together with Kotlin Coroutines and Flows in order to maintain locally available application data.

- **Gamified Student Score System**

Matrix Market will introduce a gamified Student Score system which is designed to encourage trustworthy and responsible participation within the marketplace. Users are able to earn reputational points through certain activities like completing verified transactions, maintaining their student verification status, responding consistently to other users etc.

- **Verified Trade Badges and Trust Tiers**

As an extension, Matrix Market will provide badges and trust tiers that will visually communicate positive user behaviour. So, users who consistently complete verified transactions or maintain strong response rates are able to progress through different trust levels.

- **Student-Focused Skill and Service Collaboration**

This feature will provide students with opportunities to connect with peers who are able to offer relevant academic or practical skills.

Detailed Requirements Specification

1. User Authentication & Account Management

- SSO Registration & Login: The system must be able to allow its users to register and log in using Single Sign-On (SSO). Each account has to have a unique UserID that links the user's profile, listings, messages, transactions, and reputation data.
- Biometric Authentication: The application must support fingerprint and/or facial recognition through the device's biometric authentication capabilities.
- User Profile: Users have to be able to view and manage their profile information.

2. Marketplace & Academic Resource Management

- Create and Manage Listings: Authenticated users must be able to create listings containing a title, description, price, category, optional images, and relevant academic information. Users must also be able to edit and remove listings as well.
- Search & Filtering: Users must be able to search and filter listings by keywords, category, module code, campus, price etc.
- ISBN Scanner: The application must provide an ML Kit camera-based ISBN barcode scanner that allows students to scan physical textbooks.

3. Real-Time Communication & Notifications

- In-App Messaging: Users must be able to communicate through real-time in-app messaging.
- Push Notifications: The system must provide push notifications for new messages, marketplace updates, achievement notifications etc.

4. Offline Mode & Synchronisation

- Local Storage: The application must use Room Database for locally stored application data required for offline functionality.
- Offline Access: Users must be able to access previously synchronised marketplace information while offline.
- Synchronisation: Offline changes must be stored locally with a pending synchronisation status and automatically synchronise with the REST API when connectivity is restored.
- Data Integrity: The system must prevent data loss and duplicate records during synchronisation and appropriate feedback should be providing should synchronisation fail.

5. REST API & Database

- REST API: The Android application must communicate with a REST API created specifically for Matrix Market.
- Backend Database: The REST API must connect to a server-side database containing the users, listings, academic categories, messages, student scores, and other required data.
- Authentication & Security: Protected API endpoints must require appropriate authentication and authorisation.
- Data Validation: The API must validate incoming data before storing it.
- HTTPS: Communication between the Android application and API must use secure HTTPS connections.
- Local Database: The application will maintain a local Room database containing the data required for offline functionality.
- Core Data Entities: The database should include entities like Users, Listings, Categories, Images, Messages, Conversations, UserBadges, StudentScoreHistory.

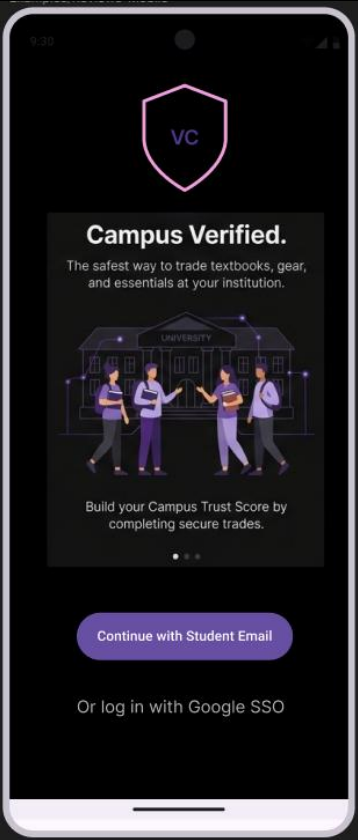
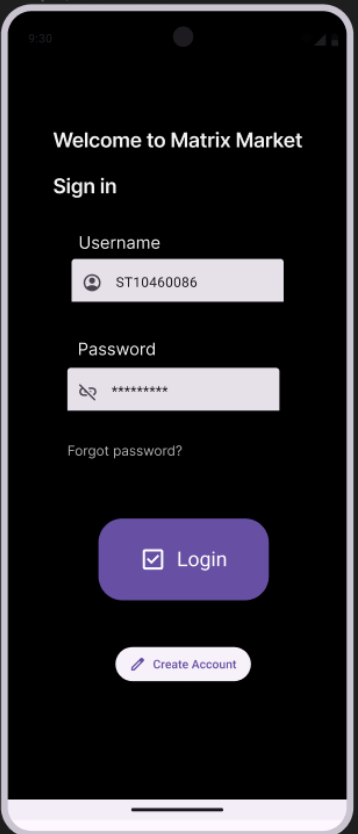
6. Gamification & Student Trust

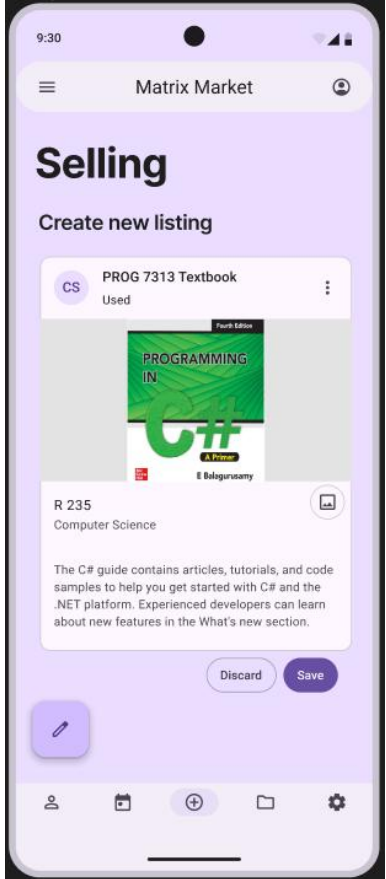
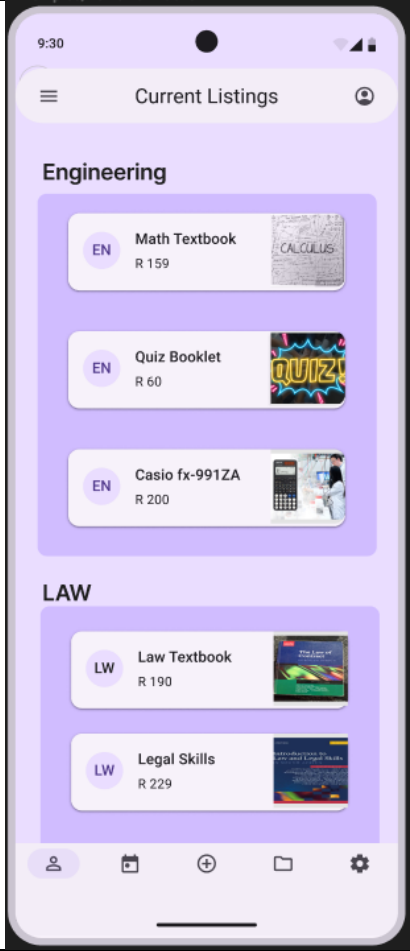
- Student Score: Each user must have a Student Score that represents their positive participation within the marketplace.
- Points: Users can earn points for activities like verified transactions, positive marketplace behaviour etc.
- Badges: The system must provide unlockable badges for defined achievements earned.
- Trust Tiers: Users can progress through defined trust levels based on their Student Score and marketplace activity.
- Profile Display: Student Scores and earned badges must be visible on the user's profile.

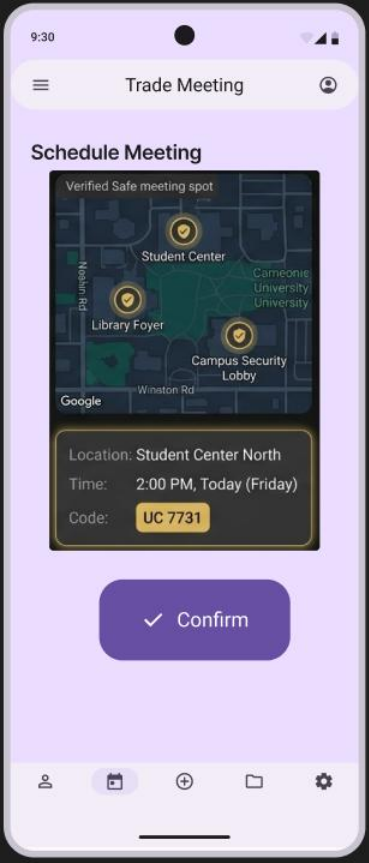
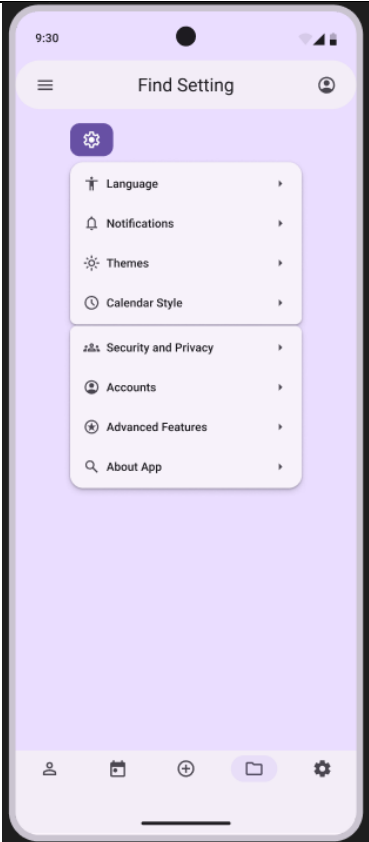
7. Settings, Languages & User Experience

- Application Settings: Users must be able to manage biometric authentication, language preferences, notification preferences, privacy settings, and account options.
- Multi-Language Support: The application must support English and at least two South African languages.
- Native Android Interface: The application must use Kotlin and Jetpack Compose
- Accessibility & Usability: The marketplace functionality should remain simple and accessible across supported Android devices.

User Interface Design

Mockup	Description
	The launch and onboarding interface serve as the primary gateway when a user opens the Matrix Market application. Designed with a sleek dark aesthetic featuring neon purple accents and high-contrast typography, the screen introduces the core value proposition a safe, institution-verified student trading, in a clean and professional manner.
	The Login screen provides a Sign-In interface serves as the primary authentication portal for returning users and registered students accessing Matrix Market. Designed with a high-contrast dark theme, light muted input containers, and prominent royal purple call-to-action buttons, this screen facilitates a secure, frictionless login experience while maintaining a minimalistic visual structure.

	The Selling page allows students to publish or preview new items such as textbooks, tech gear, or residence essentials to the Matrix Market platform. Designed using a light lavender theme with dark text and purple action elements, this screen prioritises visual clarity by displaying a live card preview of the item before finalising the post.
	The Current Listings serves as the primary browsing directory for Matrix Market, allowing students to explore available items organised logically by academic discipline. Utilising a light lavender theme with dark bold headers and rounded container grouping, this screen uses elements of various design elements so buyers can locate relevant textbooks and equipment quickly.

	The Trade Meeting interface serves as the primary safety feature for coordinating physical exchanges on campus within Matrix Market. Set against a light lavender background with high-contrast dark accents and a dark-mode interactive map card, this screen allows students to designate verified campus locations, lock in exchange times, and generate secure verification codes to prevent fraudulent trades.
	The Application Settings interface serves as the central management hub for Matrix Market, giving students granular control over app behaviour, localised preferences, account security, and notification triggers. Set against a clean lavender backdrop with a purple gear icon header, the screen organises options into distinct, elevated card containers with clear right-chevron indicators (>) for intuitive navigation.

Navigation Diagram

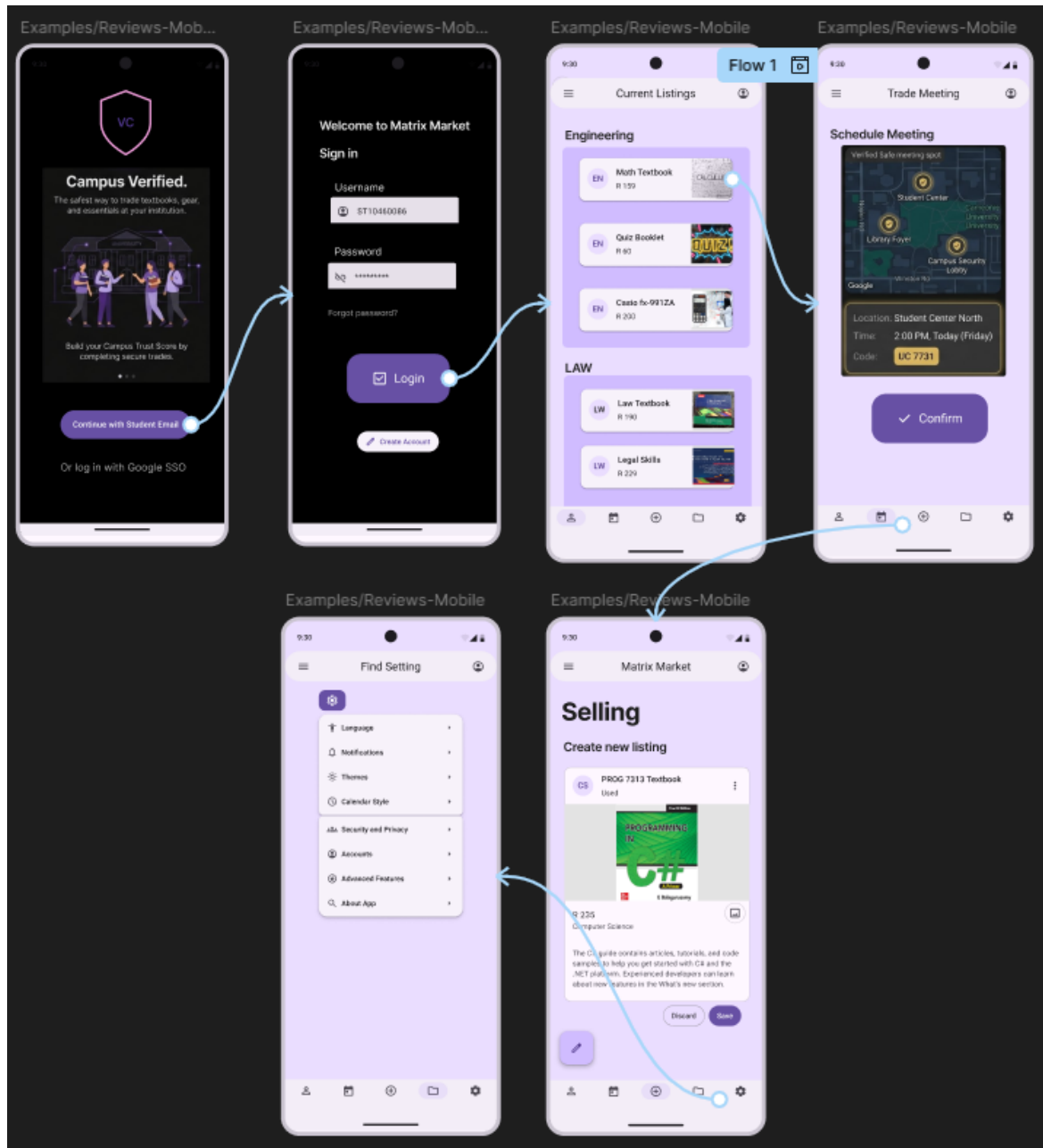


Figure 2: Navigation Flow Designed on Figma (2026)

Rest API and Back-End Architecture

System Architecture Diagram

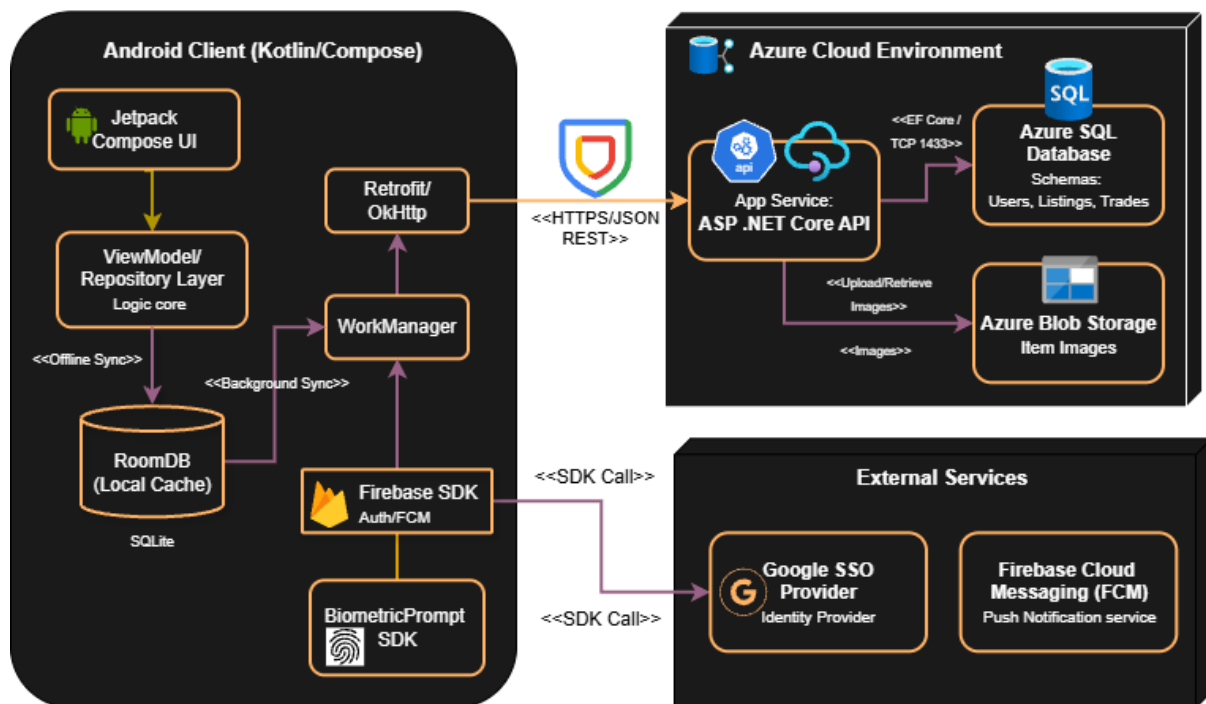


Figure 3: System Architecture Diagram designed on Draw io

Android Client: Built with Kotlin and Jetpack Compose (MVVM pattern using StateFlow and ViewModel). Retrofit handles network calls, and Room DB handles local caching.

Backend API: C# ASP.NET Core Web API hosted on Azure App Service, utilising Entity Framework Core for ORM operations and JWT (JSON Web Tokens) for stateless session security.

REST API Endpoint Specification (ASP.NET Core)

Authentication & Users

- **POST /api/v1/auth/register:** Registers student account with validated .ac.za or campus email domain.
- **POST /api/v1/auth/login:** Authenticates user and returns a JWT Bearer Token.
- **GET /api/v1/users/{userId}/profile:** Retrieves user profile, Karma Points, and awarded Trust Badges.

Item Listings

- **GET /api/v1/listings?category={id}&search={term}&type={sell|lend|trade}:** Fetches active marketplace listings with optional filtering.
- **POST /api/v1/listings:** Creates a new item listing. (Requires Auth)

- **GET /api/v1/listings/{id}**: Returns listing details, seller metadata, and trust metrics.

Trade Meetings & Gamification

- **POST /api/v1/offers**: Submits a purchase, lend, or trade offer for an active listing.
- **POST /api/v1/trades/schedule**: Creates a trade meeting entry with a system-generated 6-digit confirmation code.
- **POST /api/v1/trades/{meetingId}/confirm**: Validates confirmation code upon physical exchange; updates seller Karma Points and triggers badge evaluation.

Required Android SDKs & Libraries

To fulfil the project requirements for Part 1, Part 2, and the Final POE, these SDK's and libraries will be used in the app:

- Retrofit & OkHttp (com.squareup.retrofit2:retrofit, com.squareup.okhttp3:logging-interceptor): For HTTP client communications and JSON parsing via Gson or Kotlinx Serialization (GeeksforGeeks, 2021).
- Room Persistence Library (androidx.room:room-runtime, androidx.room:room-ktx, androidx.room:room-compiler): Handles SQLite offline storage, cached listing persistence, and local draft syncing.
- Biometric Framework (androidx.biometric:biometric): Required for the BiometricPrompt framework during high-value transactions.
- Firebase SDKs (com.google.firebase:firebase-auth, com.google.firebase:firebase-messaging): Handles Google SSO authentication and real-time push notifications.
- Coil (io.coil-kt:coil-compose): Asynchronous image loading optimised for Jetpack Compose UI (Domen Lanišnik, 2025).
- WorkManager (androidx.work:work-runtime-ktx): Manages background sync jobs when transitioning from offline mode to online network connectivity (Developers, 2025).

Data Models and Schema Definitions

Relational Schema (SQL Server / Azure SQL)

CREATE TABLE Users (

 UserId INT IDENTITY(1,1) PRIMARY KEY,
 StudentEmail VARCHAR(150) UNIQUE NOT NULL,
 PasswordHash VARCHAR(MAX) NOT NULL,
 FullName VARCHAR(100) NOT NULL,
 KarmaPoints INT DEFAULT 0,
 CreatedAt DATETIME2 DEFAULT GETUTCDATE()

);

CREATE TABLE Categories (

 CategoryId INT IDENTITY(1,1) PRIMARY KEY,
 CategoryName VARCHAR(50) NOT NULL,
 Description VARCHAR(200)

);

CREATE TABLE Listings (

 ListingId INT IDENTITY(1,1) PRIMARY KEY,
 SellerId INT FOREIGN KEY REFERENCES Users(UserId),
 CategoryId INT FOREIGN KEY REFERENCES Categories(CategoryId),
 Title VARCHAR(100) NOT NULL,
 Description TEXT NOT NULL,
 Price DECIMAL(18,2) NOT NULL,
 ListingType VARCHAR(20) CHECK (ListingType IN ('Sell', 'Trade', 'Lend')),
 ImageUrl VARCHAR(MAX),
 Status VARCHAR(20) DEFAULT 'Active', -- Active, Reserved, Sold
 CreatedAt DATETIME2 DEFAULT GETUTCDATE()

);

```
CREATE TABLE Offers (  
    OfferId INT IDENTITY(1,1) PRIMARY KEY,  
    ListingId INT FOREIGN KEY REFERENCES Listings(ListingId),  
    BuyerId INT FOREIGN KEY REFERENCES Users(UserId),  
    OfferedPrice DECIMAL(18,2),  
    Status VARCHAR(20) DEFAULT 'Pending', -- Pending, Accepted, Declined, Completed  
    CreatedAt DATETIME2 DEFAULT GETUTCDATE()  
);
```

```
CREATE TABLE TradeMeetings (  
    MeetingId INT IDENTITY(1,1) PRIMARY KEY,  
    OfferId INT FOREIGN KEY REFERENCES Offers(OfferId),  
    CampusSpot VARCHAR(100) NOT NULL, -- e.g., "Library Foyer"  
    MeetingTime DATETIME2 NOT NULL,  
    ConfirmationCode VARCHAR(6) NOT NULL,  
    IsConfirmed BIT DEFAULT 0  
);
```

```
CREATE TABLE TrustBadges (  
    BadgeId INT IDENTITY(1,1) PRIMARY KEY,  
    UserId INT FOREIGN KEY REFERENCES Users(UserId),  
    BadgeType VARCHAR(50) NOT NULL, -- e.g., "Top Seller", "Fast Responder"  
    AwardedAt DATETIME2 DEFAULT GETUTCDATE()  
);
```

UML Class Diagram

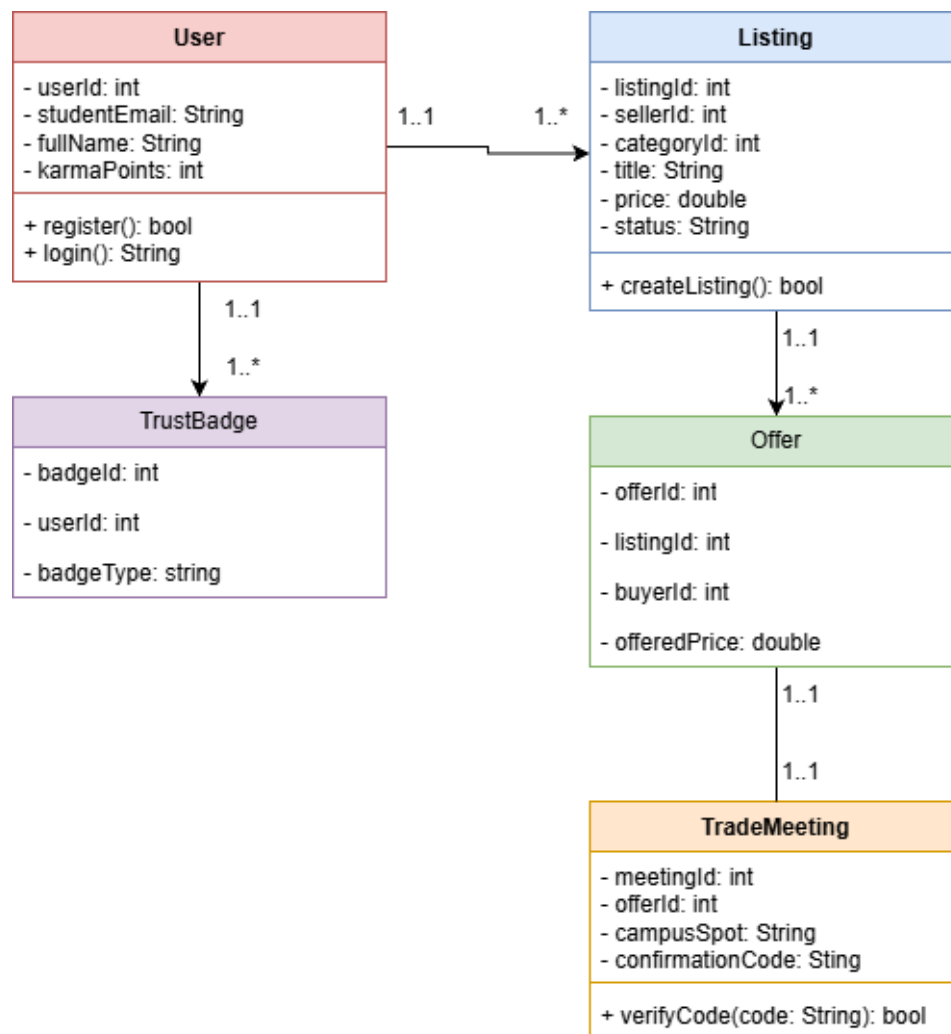


Figure 4: UML Class Diagram designed on Draw io

Project Plan

Milestones – 1.3.4, 2.2.3, 2.3.4, 4.2.1, 4.3.2, 4.3.4, 4.3.5, 5.4.1, 5.5.1, 5.5.3

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
0		Matrix Market App	92 days?	Mon 26/07/20	Tue 26/11/24			No
1		Project Planning & Management	11 days?	Mon 26/07/20	Mon 26/08/03			No
1.1		Project Charter and Scope	4 days?	Mon 26/07/20	Thu 26/07/23			No
1.1.1		Define Project Objectives	2 days?	Mon 26/07/20	Tue 26/07/21		Project Manager	No
1.1.2		Identify Stakeholders	1 day?	Wed 26/07/22	Wed 26/07/22	4	Project Manager, Research Analyst	No
1.1.3		Prepare Charter and secure approval	1 day?	Thu 26/07/23	Thu 26/07/23	5	Project Manager	No
1.2		Schedule Planning & Team Formation	3 days?	Fri 26/07/24	Tue 26/07/28			No
1.2.1		Set Milestone Schedule	1 day?	Fri 26/07/24	Fri 26/07/24	6	Project Manager	No
1.2.2		Define Deliverables	1 day?	Mon 26/07/27	Mon 26/07/27	8	Project Manager	No
1.2.3		Assign Developers, Researchers & Testers	1 day?	Tue 26/07/28	Tue 26/07/28	9	Project Manager	No
1.3		Research	4 days?	Wed 26/07/29	Mon 26/08/03			No

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
1.3.1		Identify Three Budgeting Apps	1 day?	Wed 26/07/29	Wed 26/07/29	10	Developer Researcher, Research Analyst	No
1.3.2		Conduct Research and Detailed Analysis for these apps	1 day?	Thu 26/07/30	Thu 26/07/30	12	Developer Researcher, Systems Architect	No
1.3.3		Identify Innovative Features	1 day?	Fri 26/07/31	Fri 26/07/31	13	Developer Researcher, Mobile App Developer, UI/UX Designer	No
1.3.4		Compile Research Documentation	1 day?	Mon 26/08/03	Mon 26/08/03	14	Developer Researcher, Research Analyst	Yes
2		Requirements and Design	15 days?	Tue 26/08/04	Mon 26/08/24			No
2.1		Requirements Gathering	3 days?	Tue 26/08/04	Thu 26/08/06			No
2.1.1		Document Functional Requirements	1 day?	Tue 26/08/04	Tue 26/08/04	15	Developer Researcher, Mobile App Developer, Research Analyst	No
2.1.2		Document Features (Non-Functional Requirements)	1 day?	Wed 26/08/05	Wed 26/08/05	18	Developer Researcher, Mobile App Developer, Research Analyst	No

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
2.1.3	✓	Validate with Stakeholders	1 day?	Thu 26/08/06	Thu 26/08/06	19,18	Project Manager, Research Analyst	No
2.2	✓	UI/UX Design	4 days?	Fri 26/08/07	Wed 26/08/12			No
2.2.1	✓	Wireframes, Mockups & Navigation	1 day?	Fri 26/08/07	Fri 26/08/07	20	Backend Developer, Mobile App Developer, UI/UX Designer	No
2.2.2	✓	UI Design on Figma	2 days	Mon 26/08/10	Tue 26/08/11	22	Mobile App Developer, UI/UX Designer	No
2.2.3	✓	Interactive Prototype on Figma	1 day?	Wed 26/08/12	Wed 26/08/12	22,23	Quality Assurance Engineer UI/UX Designer	Yes
2.3	✓	First Evaluation	8 days?	Thu 26/08/13	Mon 26/08/24			No
2.3.1	✓	Meeting with Stakeholder	1 day?	Thu 26/08/13	Thu 26/08/13	24	Mobile App Developer, Project Manager, UI/UX	No
2.3.2	✓	Address Stakeholder Feedback	3 days	Fri 26/08/14	Tue 26/08/18	26	Project Manager	No
2.3.3	✓	Documentation & Evaluation	3 days	Wed 26/08/19	Fri 26/08/21	27	Developer Researcher, Mobile App Developer, Pi	No
2.3.4	✓	Submission of Documentation to Stakeholder	1 day?	Mon 26/08/24	Mon 26/08/24	28	Project Manager	Yes

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
3		App Prototype Development	17 days?	Tue 26/08/25	Wed 26/09/16			No
3.1		Backend Development	6 days?	Tue 26/08/25	Tue 26/09/01			No
3.1.1		Database Schema Design	2 days	Tue 26/08/25	Wed 26/08/26	29	Backend Developer, Developer Researcher, M	No
3.1.2		GitHub Repository Creation	1 day?	Thu 26/08/27	Thu 26/08/27	32	Backend Developer	No
3.1.3		Project Creation in Android Studio	1 day?	Fri 26/08/28	Fri 26/08/28	33	Backend Developer, Mobile App Developer	No
3.1.4		Implement User Authentication	2 days	Mon 26/08/31	Tue 26/09/01	34	Backend Developer, Mobile App Developer, U	No
3.2		Mobile App Development	11 days?	Wed 26/09/02	Wed 26/09/16			No
3.2.1		Android Development	2 days	Wed 26/09/02	Thu 26/09/03	35	Backend Developer, Mobile App Developer, S	No
3.2.2		Implement Core Features	3 days	Fri 26/09/04	Tue 26/09/08	37	Backend Developer, Mobile App Developer, U	No
3.2.3		Implement Optional Features	2 days	Wed 26/09/09	Thu 26/09/10	38	Backend Developer, Mobile App Developer, Systems Architect, UI/UX	No

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
3.2.4		Implement Gamification Elements	2 days	Fri 26/09/11	Mon 26/09/14	39	Backend Developer, Developer Researcher, Mobile App Developer, Rv	No
3.2.5		Configure Offline Mode	1 day?	Tue 26/09/15	Tue 26/09/15	40	Backend Developer, Mobile App Developer	No
3.2.6		Emulator Configuration	1 day?	Wed 26/09/16	Wed 26/09/16	41	Mobile App Developer	No
4		Testing And Quality Assurance	13 days?	Thu 26/09/17	Mon 26/10/05			No
4.1		Unit Testing	4 days?	Thu 26/09/17	Tue 26/09/22			No
4.1.1		Write and automate test cases	1 day?	Thu 26/09/17	Thu 26/09/17	42	Backend Developer, Mobile App Developer, Quality Assurance	No
4.1.2		Code reviews	1 day?	Fri 26/09/18	Fri 26/09/18	45	Backend Developer, Project Manager, Systems Architect	No
4.1.3		Fix defects	2 days	Mon 26/09/22	Tue 26/09/22	46	Backend Developer, Developer Researcher	No
4.2		User Acceptance Testing (UAT)	4 days?	Wed 26/09/23	Mon 26/09/28			No
4.2.1		Initial Prototype for user testing	1 day?	Wed 26/09/23	Wed 26/09/23	47	Backend Developer, Mobile App Developer, Project Manager, Systems Architect	Yes

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
4.2.2		Collect and analyse feedback	2 days	Thu 26/09/24	Fri 26/09/25	49	Project Manager, Research Analyst, Systems Architect	No
4.2.3		Finalise release candidate	1 day?	Mon 26/09/28	Mon 26/09/28	50	Project Manager, Quality Assurance Engineer	No
4.3		GitHub and Submission Requirements	5 days?	Tue 26/09/29	Mon 26/10/05			No
4.3.1		Create Demonstration Video	1 day?	Tue 26/09/29	Tue 26/09/29	51	Developer Researcher, Mobile App Developer, Systems Architect	No
4.3.2		Generate and Build APK File	1 day?	Wed 26/09/30	Wed 26/09/30	53	Backend Developer, Mobile App Developer	Yes
4.3.3		Upload Necessary Files to GitHub	1 day?	Thu 26/10/01	Thu 26/10/01	54	Backend Developer, Project Manager	No
4.3.4		Create README File	1 day?	Fri 26/10/02	Fri 26/10/02	55	Backend Developer, Mobile App Developer	Yes

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
4.3.5		Provide Stakeholder with GitHub Repo Link	1 day?	Mon 26/10/05	Mon 26/10/05	56,53,54,55	Project Manager	Yes
5		Final App Development	20 days?	Tue 26/10/06	Mon 26/11/02			No
5.1		Online Database Deployment	4 days?	Tue 26/10/06	Fri 26/10/09			No
5.1.1		Create Firebase Account and Configuration	1 day?	Tue 26/10/06	Tue 26/10/06	57	Backend Developer, Project Manager, Systems Architect	No
5.1.2		API Configuration	1 day?	Wed 26/10/07	Wed 26/10/07	60	Backend Developer, Mobile App Developer	No
5.1.3		Adjust Code to use online Database	2 days	Thu 26/10/08	Fri 26/10/09	61	Backend Developer, Mobile App Developer	No
5.2		App Enhancement	5 days?	Mon 26/10/13	Fri 26/10/16			No
5.2.1		Graph Implementation	2 days	Mon 26/10/12	Tue 26/10/13	62	Backend Developer, Mobile App Developer, U	No
5.2.2		Visual Enhancements	2 days	Wed 26/10/14	Thu 26/10/15	64	Backend Developer, Developer Researcher, M	No

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
5.2.3		Improve Goals Overview	1 day?	Fri 26/10/16	Fri 26/10/16	65	Backend Developer, Mobile App Developer, Quality Assurance	No
5.3		GitHub Changes	3 days?	Mon 26/10/19	Wed 26/10/21			No
5.3.1		Verification of Automated Testing	1 day?	Mon 26/10/19	Mon 26/10/19	66	Backend Developer, Mobile App Developer, Quality Assurance Engineer	No
5.3.2		Update README File	1 day?	Tue 26/10/20	Tue 26/10/20	68	Mobile App Developer	No
5.3.3		Continue Automated Unit Testing	1 day?	Wed 26/10/21	Wed 26/10/21	69	Backend Developer, Mobile App Developer, Quality Assurance Engineer	No
5.4		Deployment	4 days?	Thu 26/10/22	Tue 26/10/27			No
5.4.1		Export APK to Android Device	1 day?	Thu 26/10/22	Thu 26/10/22	70	Backend Developer, Mobile App Developer, Quality Assurance	Yes
5.4.2		Evaluate App performance	1 day?	Fri 26/10/23	Fri 26/10/23	72	Mobile App Developer, Project Manager, Quality Assurance	No

WBS	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
5.4.3		Review with Stakeholders	1 day?	Mon 26/10/26	Mon 26/10/26	73	Developer Researcher, Project Manager, Researcher	No
5.4.4		Address and Fix potential issues	1 day?	Tue 26/10/27	Tue 26/10/27	74	Backend Developer, Mobile App Developer, Quality Assurance Engineer	No
5.5		Final Submission Requirements	4 days?	Wed 26/10/28	Mon 26/11/02			No
5.5.1		Create Demonstration Video on Android Device	1 day?	Wed 26/10/28	Wed 26/10/28	75	Backend Developer, Mobile App Developer, Systems Architect	Yes
5.5.2		Compile Documentation	1 day?	Thu 26/10/29	Thu 26/10/29	77	Backend Developer, Mobile App Developer, Project Manager	No
5.5.3		Provide Stakeholder with Submission Links and Documents	1 day?	Fri 26/10/30	Fri 26/10/30	78	Project Manager	Yes
5.5.4		Post Launch Monitoring	1 day?	Mon 26/11/02	Mon 26/11/02	78,79	Backend Developer, Mobile App Developer, Quality Assurance Engineer	No

Conclusion

The research and planning for Matrix Market have established a clear foundation for implementation. The comparative evaluation of Gumtree, Facebook Marketplace and StudentVIP have identified key limitations in existing platforms and these findings helped with the proposed design choices for Matrix Market. The requirements, user interface, navigation, data structures, API architecture and system components have now been sufficiently defined to provide a clear implementation roadmap. The proposed architecture also establishes separation between the mobile application, local storage, API and backend database, which will support maintainability and scalability in the future.

Reference List / Bibliography

Africa, G. S., 2026. *Gumtree SA: South Africa's Popular Classified Marketplace*. [Online] Available at: <https://support.gumtree.co.za/portal/en/kb/articles/about-us> [Accessed 22 August 2026].

Australia, S. S., 2026. *StudentVIP: Australia's Student Textbook and Campus Marketplace*. [Online] Available at:<https://studentvip.com.au/textbooks>](<https://studentvip.com.au/textbooks>) [Accessed 22 August 2026].

Developers (2025). *Getting started with WorkManager*. [online] Android Developers. Available at: <https://developer.android.com/develop/background-work/background-tasks/persistent/getting-started> [Accessed 13 Aug. 2026].

Domen Lanišnik (2025). *Loading images with coil in compose multiplatform*. [online] Medium. Available at: <https://medium.com/@domen.lanisnik/loading-images-with-coil-in-compose-multiplatform-4c94e16a06d7> [Accessed 14 Aug. 2026].

GeeksforGeeks (2021). *Serialize and deserialize JSON using GSON library in android*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/kotlin/serialize-and-deserialize-json-using-gson-library-in-android/> [Accessed 12 Aug. 2026].